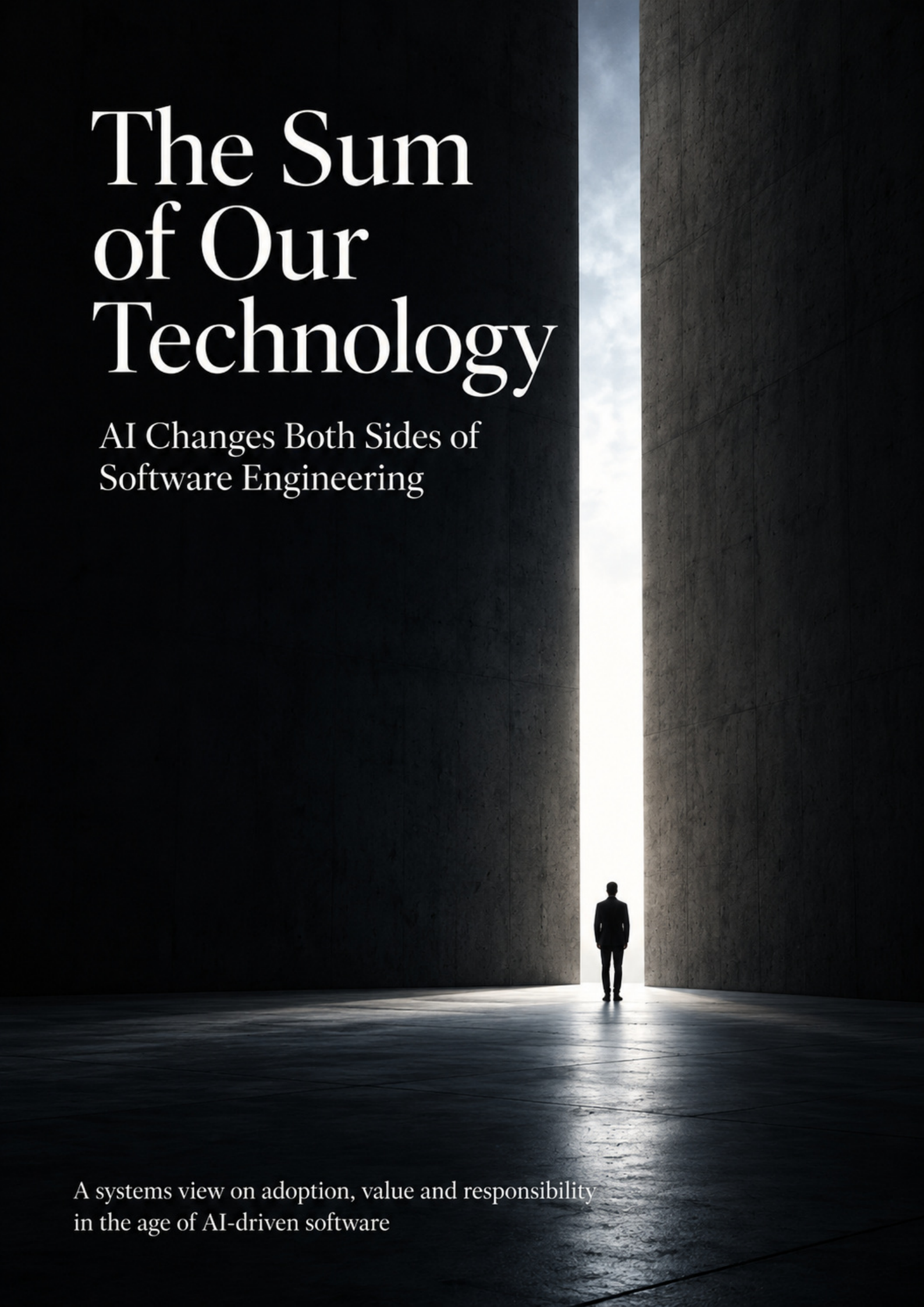

The Sum of Our Technology
AI Changes Both Sides of Software Engineering
A systems view on adoption, value and responsibility in the age of AI-driven software

The Sum of Our Technology

AI Changes Both Sides of Software Engineering

An agent opens a pull request. The tests are green. The explanation is fluent. Another arrives before anyone has understood the first.

The dashboard calls this progress. The person handling the next incident has a different question.

What if not?

What if faster generation does not improve delivery—or leaves us unable to recover what we built? What if the person entrusted with the final decision cannot meaningfully make it?

My talk at UA Online PMDay 2026 Autumn connects two shifts: AI changes **how we build software**, and model judgment changes **what software does at runtime**. We can delegate more work. We still have to own the consequences.

Slides (PDF) [1] · Recording (Ukrainian) [2]

When ideas outlive their authors

Writing let ideas survive their authors. Printing let them cross borders and centuries. Research on printing and the Reformation [3] connects that circulation with consequences beyond book production. Technology changes what people can do together, not just the speed of a task.

Large language models extend delegation to interpretation, comparison, design, implementation and explanation. Delegating thinking describes that practical change, not a claim about consciousness.

Who builds the software and what it does are separate questions. An agent can build a conventional application; a human team can build a product that delegates decisions to a model. Both shifts change what must remain dependable around the technology.

Stanisław Lem captured the reciprocal obligation:

> “The more technology takes care of us, the more carefully we must take care of it.”

— Stanisław Lem; translated from Polish for this article. Publisher-supplied excerpt and attribution: [4].

A demonstration shows that a capability can work. Engineering asks what its use requires of the larger system, including when it fails.

I. How We Build

We accelerated a step. What did the system gain?

The early story was spectacular. Cognition introduced Devin [5] in 2024 as the “first AI software engineer”. In March 2025, Dario Amodei forecast AI writing 90% of code within three to six months [6], potentially essentially all within a year. Those horizons have passed; the forecast is historical context here, not a verified industry-wide code share. Both accounts retained human responsibilities. Their code-production claims leave a larger question open: how much of software engineering has actually accelerated? Treating generated code as a measure of that whole activity is a category error.

The first readily accelerated activity in many deployments was **code generation**. Delivery requires a chain:

Intent → requirements → design → code → review → test → integration → deployment → operation.

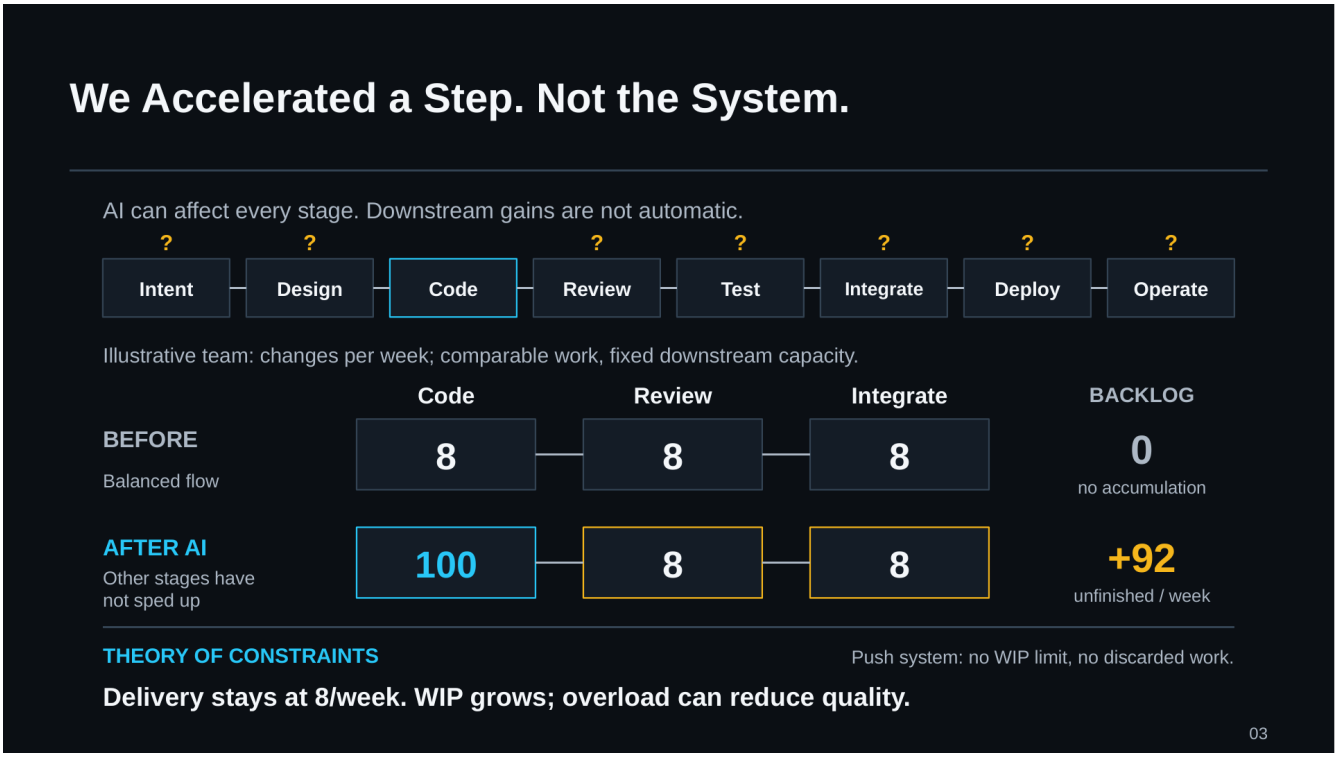
The September revision of Writing Code vs. Shipping Code [7], covering more than 500,000 GitHub developers, summarises cumulative estimates as **25.5× changed lines, 3.4× commits and 1.3× releases**. These observational, outcome-specific estimates are not a conversion funnel. Changed lines include additions and deletions; asynchronous-agent releases were not separately estimated.

A roughly 30% release gain matters. **Would you buy that gain if the price were twenty-five times as much code churn for the organisation to absorb?** The figures do not establish that trade-off for a particular team: changed lines are neither retained code size nor human reading time. They do make the price worth asking about.

The bill can arrive later. Unnecessary dependencies, duplicated logic and deferred cleanup become technical debt; harder diagnosis and slower recovery raise the total cost of ownership. If the flow overwhelms the team's ability to understand and maintain what survives it, more releases can leave the organisation with less control. DORA [8] likewise associates adoption with higher throughput—and greater instability. The gain needs to be assessed over the system's life.

Those findings do not identify a single bottleneck. A simple model illustrates one structural constraint. Suppose a team generates, reviews and integrates eight comparable changes a week. Raise generation capacity to one hundred; keep the other capacities fixed.

Admit every change, and delivery stays at eight while unfinished work grows by ninety-two a week. **Under those conditions, we become dramatically better at producing unfinished work.** Control intake instead: the queue need not grow, but review and integration still cap delivery at eight. Faster generation can save effort or waiting time without a proportional throughput gain.



Synthetic fixed-capacity illustration. Uncontrolled intake accumulates work; controlled intake bounds the queue without proportionally increasing delivery. Both regimes are explained in the text. UA Online PMDay 2026 Autumn · slide 3 · Vitalii Oborskyi. [1]
Full-page slide 3 from Vitalii Oborskyi's v35 review deck. Source-deck and third-party credits preserved; publication rights pending.

A code generator does not rebalance the chain. The synthetic example explains a possible mechanism; it does not claim to reproduce the studies' observed outcomes.

The Subprime Code Crisis [9] is my synthesis of costs shifted into review, QA, architecture, maintenance and production. The danger is booking local gains before accounting for displaced work and risk.

The narrative now follows the next bottleneck. Anthropic's Code Review announcement [10] addresses review pressure. Microsoft's Jay Parikh [11] makes it an instruction:

> **“Don’t let code review become a bottleneck”**

His wider call is **“Throw out the old playbook.”** First-pass review moves to agents; people retain architectural oversight. Elsewhere: **“Execution runs on autopilot.”** This is a proposal to redesign how review works. Its success depends on preserving both defect detection and the understanding needed for difficult decisions, even as people inspect less implementation detail.

What if not?

Suppose defect detection improves, but incident investigation becomes harder. The approval button survives; the knowledge that gave approval meaning does not. **The queue has cleared. The ownership gap has not.**

Identifying a bottleneck does not prove that its function can be removed. Changing the method may preserve that function—or improve it. The question is how the new workflow produces its necessary outcomes, including knowledge people can use when something fails.

There is important counterevidence. Anthropic reports [12] that merged Claude Code PRs more than doubled while the reverted share fell over 40% between June and September 2026. The team describes layered checks, risk-based approvals and staged release alongside these encouraging internal results—not just promises.

They establish neither causality nor portability. A lower reverted share need not mean fewer absolute reversions, and neither measure captures shared understanding or every latent defect. The team uses on-demand explainers and open working channels to share context, while acknowledging no formal process for maintaining shared understanding and no automatic tightening of approvals when incidents or reversions rise.

A new equilibrium is possible. **The surrounding redesign was part of the problem from the beginning—not an optional extra to discover after rollout.**

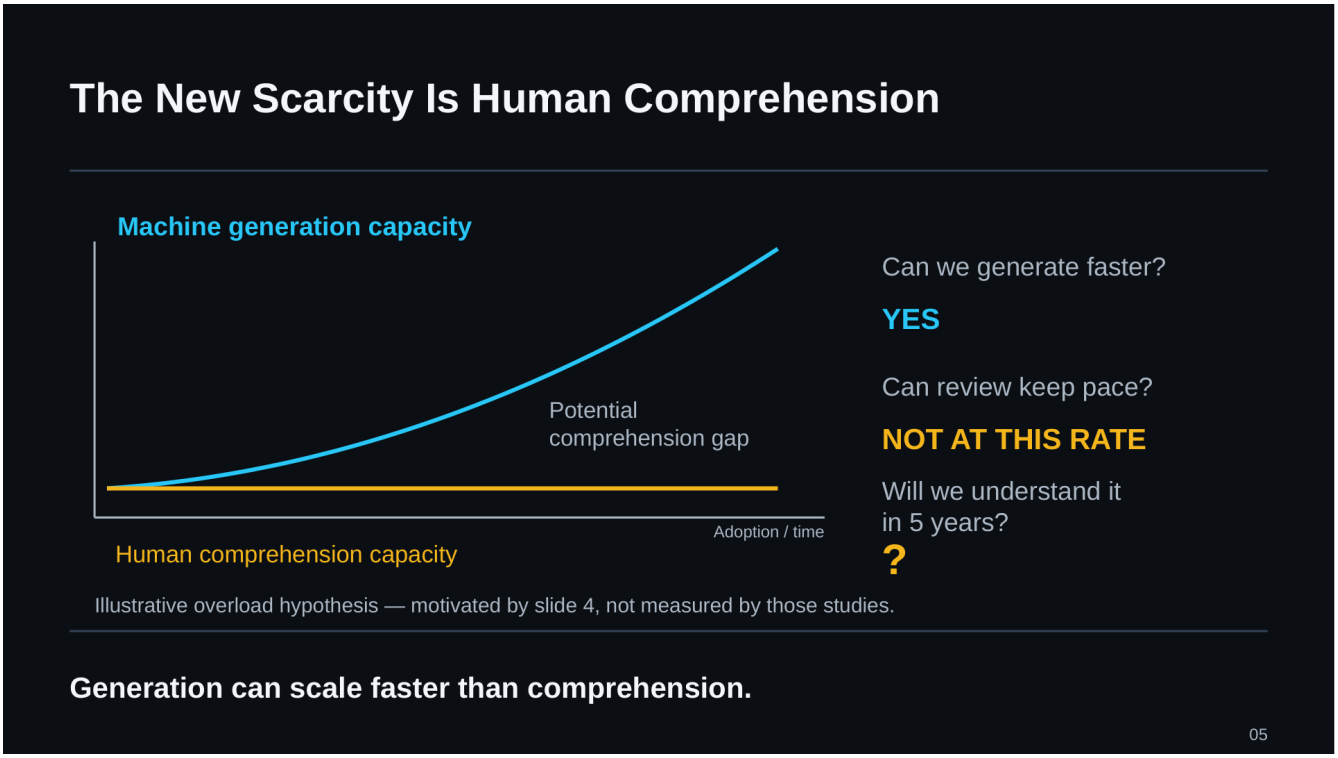
The scarce resource is understanding

The burden may be more than volume. In a longitudinal matched-control study, Agarwal, He and Vasilescu [13] estimate increases in SonarQube's Cognitive Complexity of **34.85% for agent-first repositories and 42.87% for those with prior AI-IDE exposure**. Their model controls for lines of code; this is neither per-line complexity nor measured human comprehension. GitClear [14] reports **81% more duplicated code blocks than its 2023 baseline**—an observational vendor signal, not proof of AI causation.

Complexity bears directly on the work of understanding. An empirical validation of Cognitive Complexity [32] found associations with comprehension time and perceived difficulty. A nested branch adds conditions to those a reader already has to hold in mind; a dependency adds relationships to reconstruct. Their interaction can make a small change disproportionately harder to reason about.

That does not give us a universal curve from a static score to human effort. Further empirical work [33] found that code measures alone predicted understanding poorly: the person and the task matter. Rising complexity is a warning about the effort needed to retain ownership, not a direct measurement of that effort.

The presentation's generation/comprehension gap illustrates this overload risk. Tools, abstractions and learning can raise comprehension capacity. The operating limit is reached when the team can no longer explain critical behaviour or diagnose and contain failures in time. Does our workflow preserve that ability?



An illustrative overload hypothesis, not a measured limit on cognition. Tools and learning can change the team’s comprehension capacity. UA Online PMDay 2026 Autumn · slide 5 · Vitalii Oborskyi. [1]
Full-page slide 5 from Vitalii Oborskyi's v35 review deck. Source-deck and third-party credits preserved; publication rights pending.

Review can find defects and spread knowledge of what changed and why. Bacchelli and Bird [15] identify knowledge transfer and team awareness among its outcomes. Automating one does not reproduce the others; ceremonial human approval delivers neither.

Bainbridge's Ironies of Automation [16] describes the older trap: automation leaves people the difficult exceptions while reducing the practice needed to handle them.

Responsibility does not disappear when understanding disappears. We are left responsible for something we are less capable of controlling.

The explanation will give us the understanding

Ask an LLM to explain the code. Perhaps specifications and generated documentation become the durable artifacts, implementation a replaceable detail.

What if not?

We now trust an implementation and a story about it. The story can inherit the mistake or omit the exception. Specifications, code and tests can agree because they share the same wrong assumption. Agreement is not independent verification. Even a faithful account of what the code does does not establish that this is what the business needed.

Connect explanations to exact revisions, observed traces and independent requirements. Ask a responsible maintainer, using the team's actual tools, to explain a critical decision, trace its dependencies and diagnose an agreed failure scenario. Record where missing context blocks them. Understanding is a demonstrated capability, not a quantity of Markdown.

We can always regenerate the code

Parikh's other blunt heading is **“Code is disposable”**. For a well-bounded feature [11], replacement may be cheaper than repair and the specification more durable. If implementation fails, the agent can supply a substitute.

What if not?

Customers cannot complete transactions. The agent proposes two fixes; neither restores service. A rewrite looks tempting, but existing data and integrations cannot be discarded. Who stops the retries, contains damage and establishes that recovery worked?

Code may be disposable. Your customers' data, obligations and consequences are not.

Ownership is not memorising every line. It is retaining the ability to investigate, challenge and change the system when an abstraction fails. AI can help; recovery must still work when the preferred agent does not.

Rehearse recovery with a retry limit, preserved diagnostics and an authorised owner. Record time to contain harm, time to restore service and whether data remain intact—not merely whether the service restarts.

A new equilibrium has to be demonstrated

The baseline deserves the same scrutiny. Human approval alone does not establish understanding; an established workflow does not establish control. Compare the proposed system with the one you actually operate, including its failures—not an idealised old equilibrium. Improvement over that baseline and compliance with necessary limits are separate requirements.

Try smaller changes, work-in-progress limits, specifications, AI-assisted review and clearer ownership. Compare review age, rework, stability, maintained knowledge, recovery and total cost—including ongoing maintenance and the work needed to keep dependencies and duplicated logic under control. These are candidate interventions, not universal remedies. A faster queue with slower recovery means the cost has changed address.

We Are Searching for a New SDLC Equilibrium

OLD EQUILIBRIUM	TODAY	NEW EQUILIBRIUM
Human-led flow	Generation can outpace review	Sustainable flow?

WATCH / MEASURE

FLOW & REVIEW
Review age, batch size and WIP
Lead time
CODEBASE & MAINTAINABILITY
Churn, duplication and complexity
Code / dependency growth vs value
DELIVERY & PRODUCTION
Release throughput and stability
Incidents, rollback and recovery time
TEAM, OWNERSHIP & RECOVERY
Review / on-call load and ownership
Team health; recovery without AI

EXPLORE / TEST

FLOW CONTROL
Small batches and WIP limits
Match generation to review capacity
SPEC-FIRST / DISPOSABLE CODE
Trusted specs and tests as anchors
Regenerate only when cheaper
KNOWLEDGE FOR HUMANS + AGENTS
Living docs and ownership maps
Knowledge retrieval for people + agents
AI-ASSISTED REVIEW & RECOVERY
AI review plus static and eval gates
Code + docs review; recovery drills

Candidate countermeasures — not proven fixes.

Test the gap, delivery, quality, full cost and team health.

07

Measures and candidate responses, not universally validated remedies. Faster generation alone does not establish a sustainable new equilibrium. UA Online PMDay 2026 Autumn · slide 7 · Vitalii Oborskyi. [1]

Full-page slide 7 from Vitalii Oborskyi's v35 review deck. Source-deck and third-party credits preserved; publication rights pending.

More software is not more value

Suppose the redesigned workflow ships much faster and downstream work keeps up. More ideas reach customers sooner. We expect more value.

What if not?

Features multiply; the customer's problem remains. We have shortened the journey from an assumption to an implementation without establishing that the assumption was right.

Cheap experiments should help us reject weak ideas, not skip that decision. Kohavi and colleagues [17] report that only about one-third of well-designed Microsoft experiments improved their intended metric—a finding from their setting, not a universal success rate. Deciding not to ship can be a successful outcome.

NBER [7] reports low-usage shares rising from **78% to 87%** for new iOS apps with fewer than ten ratings, and **19% to 33%** for Chrome extensions with fewer than ten downloads. These observational, first-three-month usage proxies measure neither welfare nor AI's causal effect.

METR's 349-person survey [18] reports median self-assessed speed gains of **3×** and **1.4–2× work-value gains**, depending on wording. The perceived gains matter; additional output is not their measure.

The user's capacity is part of the product

Grant the stronger case: every feature is useful, and a balanced SDLC delivers twenty to thirty times more customer-facing changes. This is a thought experiment, not NBER's release estimate. The promise is realised customer value and competitive advantage.

What if not?

A customer uses the reporting tool monthly; its workflow changes weekly. Each improvement makes sense. Each visit still requires relearning the job. Delivery has accelerated. Attention and learning time have not.

Useful capability is not yet realised value. It must be discovered, understood and incorporated into work. **The comprehension problem has crossed the product boundary—from builders to users.**

AI did not create this learning burden. It can increase how quickly we deliver changes that depend on finite human capacity. Feature-fatigue research [19] examines a related, older problem: capabilities attractive before use can undermine usability afterwards. It establishes no universal learning rate. A daily expert and an occasional user may face very different adjustment costs.

Nor does every improvement require learning: better defaults, faster responses and repaired defects can remove work. Measure successful task completion, adoption, time to benefit and support burden. Sequence changes people must learn; spend capacity on accessibility, reliability and fewer steps.

Competitive advantage is not how much change we deliver. It is how much better customers can work because of it.

Rebalancing around human capability

Rebalancing generation, review, testing and integration brings us to a problem of a higher order: how to organise the whole chain around the different speeds of machines and people. The chain reaches beyond deployment, into maintenance, incident response and the customer's working day.

People need enough understanding to own the system, make informed decisions and recover it after an incident. That does not require following every generated line. It requires access to the decisions and dependencies that matter, practice in diagnosing failure, and time to use both. Tools can expand that capacity; the workflow has to demonstrate that they do.

The sustainable gain is whatever speed we can turn into useful outcomes while retaining those abilities. Otherwise, each local bottleneck we remove carries us closer to a larger one: a system we can change faster than we can responsibly operate.

II. What We Build

When the product itself makes judgments

Model judgment now operates **inside the product**. An unsafe answer can become an operational event without a pull request.

Consider an account-recovery assistant. It interprets requests, retrieves guidance and proposes actions. More flexible than a rigid decision tree, it also determines part of the path to consequences at runtime. Even without permission to change access directly, its guidance can influence someone who has it. Read-only does not mean consequence-free.

Here the model's judgment helps perform a function with real consequences. In my Uncertainty Architecture [20] framework, that is a Thinking System [21]: at least one consequential runtime responsibility depends partly on probabilistic model judgment. The name describes implementation, not consciousness or safety.

Consider six assumptions a design may rely on—and what each requires when it fails.

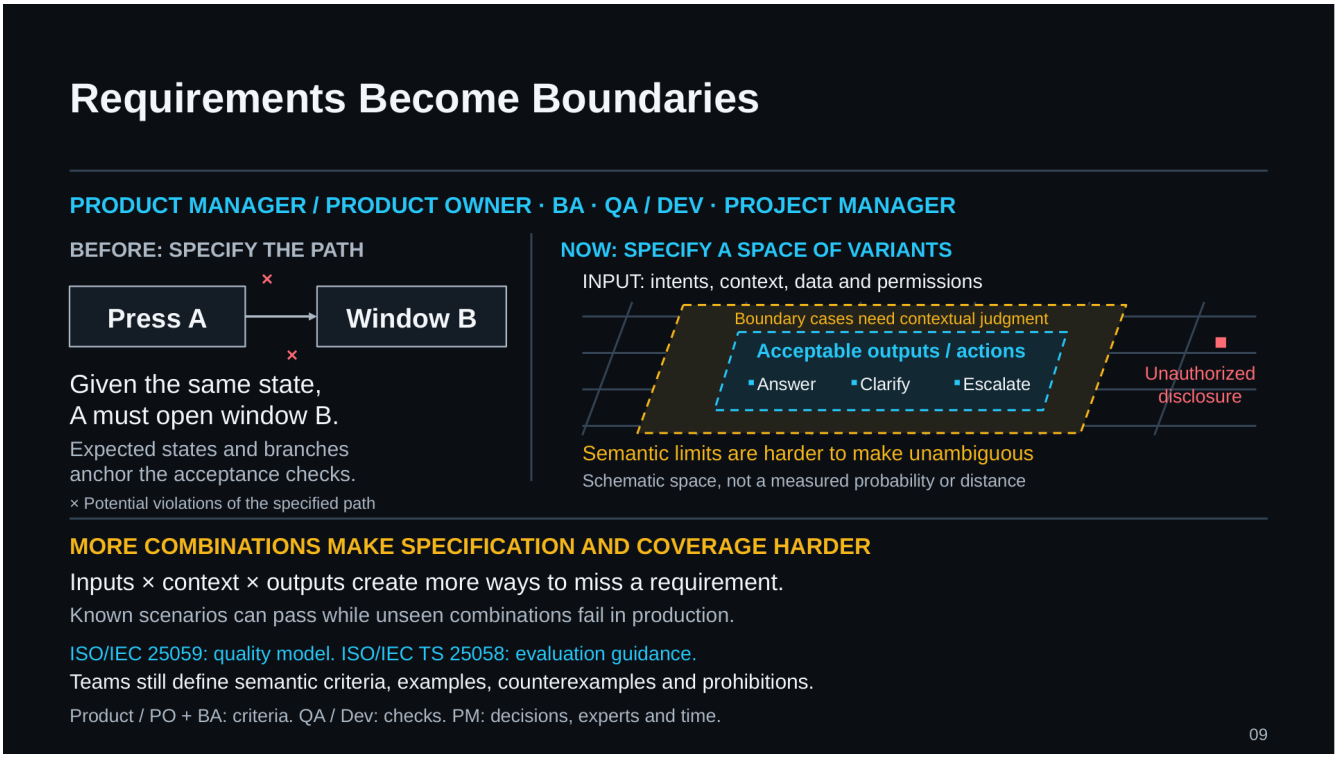
We will tell the model what it must not do

Supply approved guidance and prohibit invented steps, restricted disclosure and bypassed authorisation. Those instructions should bound the assistant's judgment.

What if not?

Conflicting documents lead it to choose the wrong guidance. Or someone makes the conflict deliberate: an attacker plants instructions in material the assistant retrieves, posing as recovery policy and urging it to skip identity checks. The assistant treats source content as authority. This hypothetical account-recovery attack follows the indirect prompt-injection mechanism demonstrated by Greshake and colleagues [22].

A prompt prohibition is not an enforced boundary. Specify when to answer, clarify or escalate. Restrict retrieval, enforce permissions outside model judgment and test hostile inputs. Retrieved text cannot grant itself policy status or expand permissions. OWASP's guidance [23] likewise emphasises trust boundaries and least privilege; no single layer guarantees immunity.



The drawing is schematic, not a probability map. Answering, clarifying or escalating can all be correct; unauthorised disclosure remains outside the boundary. UA Online PMDay 2026 Autumn · slide 9 · Vitalii Oborskyi. [1]
Full-page slide 9 from Vitalii Oborskyi's v35 review deck. Source-deck and third-party credits preserved; publication rights pending.

The evaluation will tell us whether it works

Test the requirements. The presentation uses **200 synthetic responses**:

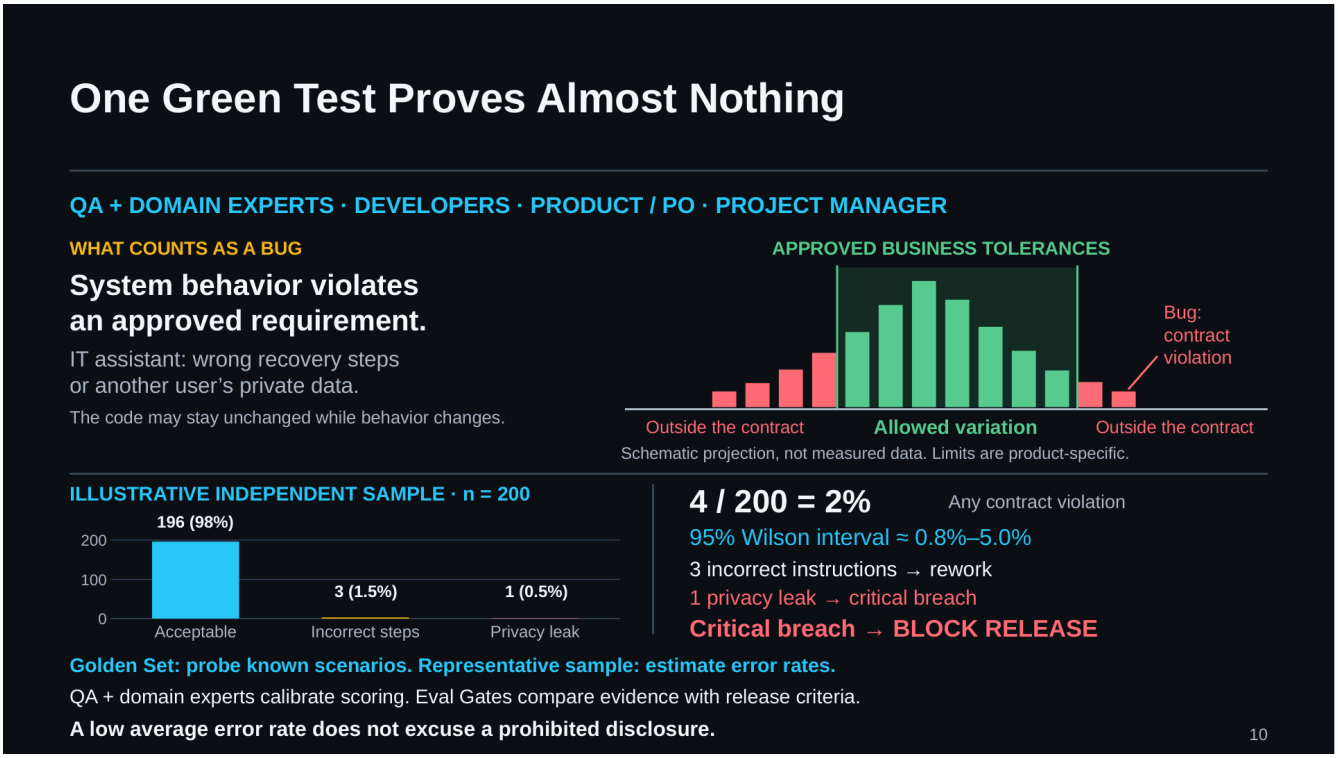
Outcome	Responses
Acceptable	196
Incorrect recovery instructions	3
Restricted information disclosed	1

Ninety-eight percent acceptable. The tempting conclusion: ready to take responsibility.

What if not?

The percentage is correct; the release decision is wrong. Under a rule prohibiting that disclosure, the candidate fails. The other responses cannot cancel the consequence.

The critical class is **1/200: 0.5% observed, with an approximate 95% Wilson interval of 0.09%–2.78%**. It quantifies uncertainty under independent, representative sampling and stable conditions—not permission. A curated Golden Set tests known requirements and difficult cases; it need not represent production. [24]



Synthetic 196 / 3 / 1 outcomes. The slide's four-failure interval concerns all failures; the article separately calculates uncertainty for the one critical disclosure. UA Online PMDay 2026 Autumn · slide 10 · Vitalii Oborskyi. [1]
Full-page slide 10 from Vitalii Oborskyi's v35 review deck. Source-deck and third-party credits preserved; publication rights pending.

Even 3,000 failure-free trials give only a one-sided 95% upper bound near 0.1% under those assumptions. [24] Rates from ordinary traffic do not establish resistance to an attacker choosing and adapting inputs. Test those threats separately.

Evaluation supplies evidence. Severity, coverage, uncertainty and enforceable limits determine what it justifies delegating.

Another model will check the first one

A second model inspects proposals, catching errors the generator misses. The combined path should be more dependable.

What if not?

Both accept the same misleading document and unsupported premise. Another invocation supplies a verdict, not independent evidence.

Knight and Leveson [25] found more coincident failures in independently implemented programs than independence predicted. This historical warning is not a measurement of modern LLM error correlation.

Test the checker against verified failures, valid outputs and shared-error cases. Measure missed errors and unnecessary blocks. Enforce permissions separately. Timeout or uncertainty must trigger a defined wait, denial or fallback—not become authorisation.

We will monitor it and roll back

Telemetry alerts the owner; rollback reverses the change. We expect correction fast enough to contain consequences.

What if not?

The alert arrives after disclosure or after an employee follows unsafe instructions. Reverting the model erases neither. **Recovery worked; the assumption that it could contain this harm did not.**

Enforce critical limits before action. Observe outputs, denials, fallbacks, reviewer queues and control health. An authorised owner needs a tested way to narrow or stop operation and verify the intervention. NIST's AI RMF [28] treats monitoring, response and recovery as operational responsibilities.

If intervention cannot beat the consequence, restrict authority, redesign or reject the path. Monitoring cannot prevent harm it only reports afterwards.

The human is an architectural dependency

A qualified person will receive the evidence, recognise the exception and intervene before harm. That is what “human in the loop” promises.

What if not?

The escalation queue is unattended at night. At peak load, reviewers cannot keep up. A tired reviewer approves a polished summary without seeing the evidence. The path exists. The capacity to operate it does not.

Parasuraman and Manzey [26] connect automation complacency to competing tasks and attention, and document automation bias in novices and experts. Experience and instructions are not sufficient guarantees. Bainbridge's trap returns: the hardest exception reaches someone whose opportunities to practise have diminished. [16]

If safe operation depends on human judgment, human capability is an architectural dependency.

Architects must account for staffed availability, competence, queue capacity, cognitive load, interruptions, evidence and the power to refuse. Measure queue wait plus decision time under normal and peak demand against the available intervention window. Operations must supply that capacity. People are not interchangeable APIs; agency and time to rebuild context belong in the design.

An escalation policy cannot staff a queue. If a human decision lies on the critical path, staffing and response capacity are operating costs and release conditions—not free resources supplied by the organisation chart.

Human-factors research grounds the problem. UA's control-loop anatomy [27] supplies my architectural framing of human response time, capacity and decision quality. We are engineering a **socio-technical stack**, not software plus an oversight footnote.

Make it concrete: without qualified approval in time, the assistant cannot change account access. Remove that permission. An unavailable reviewer must never turn waiting into approval.

That still leaves the danger of someone following unsafe advice. When no qualified reviewer is available, the application suppresses model-generated recovery advice and displays a fixed, preapproved message directing the requester to the verified recovery channel. The model cannot select another procedure, add steps or expose account-specific information. Case-specific recovery waits for the staffed process.

A bounded release can therefore provide verified routing while withholding account-specific recovery. Its evidence must cover the fixed output path, isolation from account data and the capacity of the receiving process. Broader advice or action needs its own evidence and approval.

The hypotheses outlive the release

“It passed before release.” Evidence, controls, human capacity and fallback support bounded operation. We can rely on that arrangement while its conditions hold.

What if not?

The model changes, retrieval acquires conflicting material or demand overwhelms reviewers. Routing changes at the provider. No application commit is required for yesterday's evidence to stop describing today's system.

Record conditions of reliance and reassessment triggers. Turn incidents into regression cases; retest controls and capacity; revisit authority when its justification changes. The owner must know which model, data, permissions, workload and human coverage the original evidence actually supported.

Release authorises operation under particular conditions. It cannot make those conditions permanent.

The hypotheses do not die when we release. They become conditions of operation. A belief about model behaviour, checker independence or reviewer capacity remains open to challenge for as long as operation depends on it. Evidence may strengthen it, disprove it or require a different design.

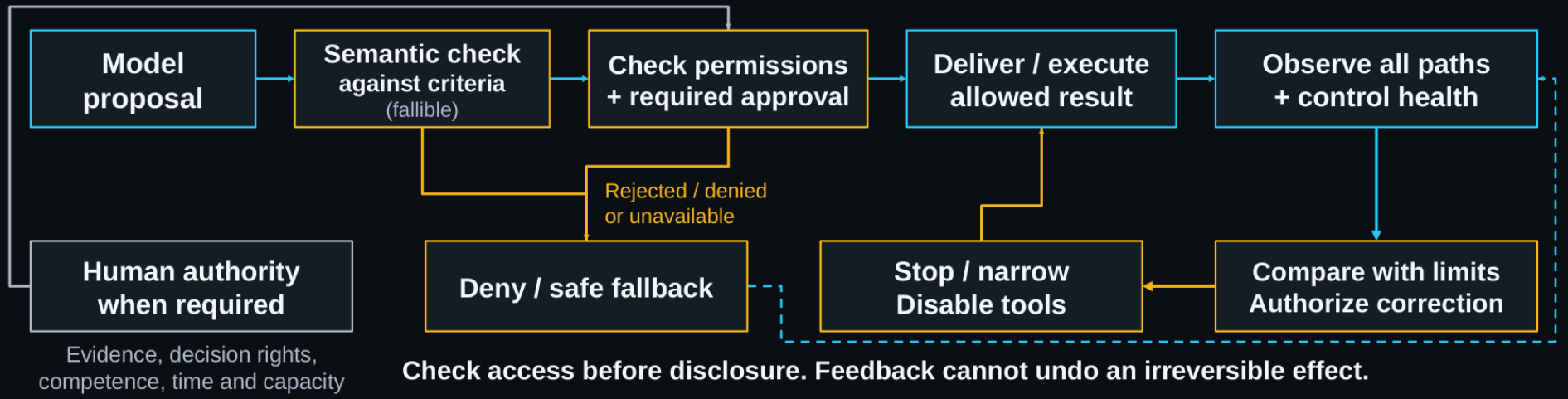
The architecture must sustain both the grounds for delegated authority and the human capability to judge when those grounds no longer hold. The people responsible need enough context, time and power to restrict, repair or stop the system. Their ability to do so is part of what we build.

Production Needs a Control Loop

Validate control effectiveness. Telemetry alone does not establish correctness.

ARCHITECT + DEV: implement · QA: verify · SERVICE OWNER: operate · PM: dependencies

IT ASSISTANT: CONTROL BEFORE ACTION AND AFTER DEPLOYMENT



LATENCY & PEOPLE

Can control act before consequence? Is human capacity real?

FULL CONTROL COST

Tokens, evaluations, retries and human review.

EXTERNAL CHANGE

Model, routing and provider assumptions can change.

No reliable, operable control path? Narrow, redesign or reject this AI path.

12

Permissions, evidence, fallible semantic checks and verified intervention form one operating path. Real human capacity and response time must be supplied and tested. UA Online PMDay 2026 Autumn · slide 12 · Vitalii Oborskyi. [1]

Full-page slide 12 from Vitalii Oborskyi's v35 review deck. Source-deck and third-party credits preserved; publication rights pending.

III. What If Not?

The person handling the next incident needs more than a green dashboard. They need to know what changed, which evidence still applies and how to stop the damage. Both shifts return us to that obligation.

In delivery, we delegate more of the work that changes the system; in the product, we delegate judgments within its operation. Either can leave people unable to explain a failure. Either can create irreversible harm. The distinction tells us where authority is delegated, not which consequences belong exclusively to each shift.

A working recovery mechanism is not the same as a recoverable consequence. Reverting software cannot retrieve disclosed information or erase an action someone took on its advice.

Small releases and rapid learning remain valuable. Canary releases [29] make exposure, evaluation and the decision to expand explicit. The same discipline asks whether we can detect trouble and intervene before the unacceptable consequence.

Agile's principles [30] connect agility with technical excellence, good design and a sustainable pace. But a release cadence cannot supply the whole analysis: how dependencies interact, how understanding and skills evolve, how maintenance costs accumulate, and whether intervention will still work after the original team moves on. **“We will iterate” is not an answer to a consequence we cannot reverse.** Iteration belongs within an account of the system's life and responsibilities.

When adoption outpaces verification, containment and recovery, the cost of learning leaves the experiment. The people and organisations exposed to it pay.

Vendors owe adopters usable evidence, clear limits and visibility into changes that affect reliance on their systems. Their specialists do not staff your incident response. Their risk profile is not yours: different commitments, resources and exposure can make a tolerable experiment for them an unacceptable one for your customers. **Their confidence is not your risk acceptance.**

Lem's warning applies across that chain:

> “One cannot simultaneously make discoveries and avoid taking responsibility for their consequences.”

— Stanisław Lem, quoted and translated by Bogna Konior; source and edition details: [31].

The more power we delegate, the more rigor we owe the systems that will bear its consequences. That obligation belongs to those supplying, approving, designing and operating the adoption. It includes funding the human capability on which control depends.

I want this to work. Less drudgery, better verification and previously unaffordable software are worth pursuing. Start with a decision small enough to justify:

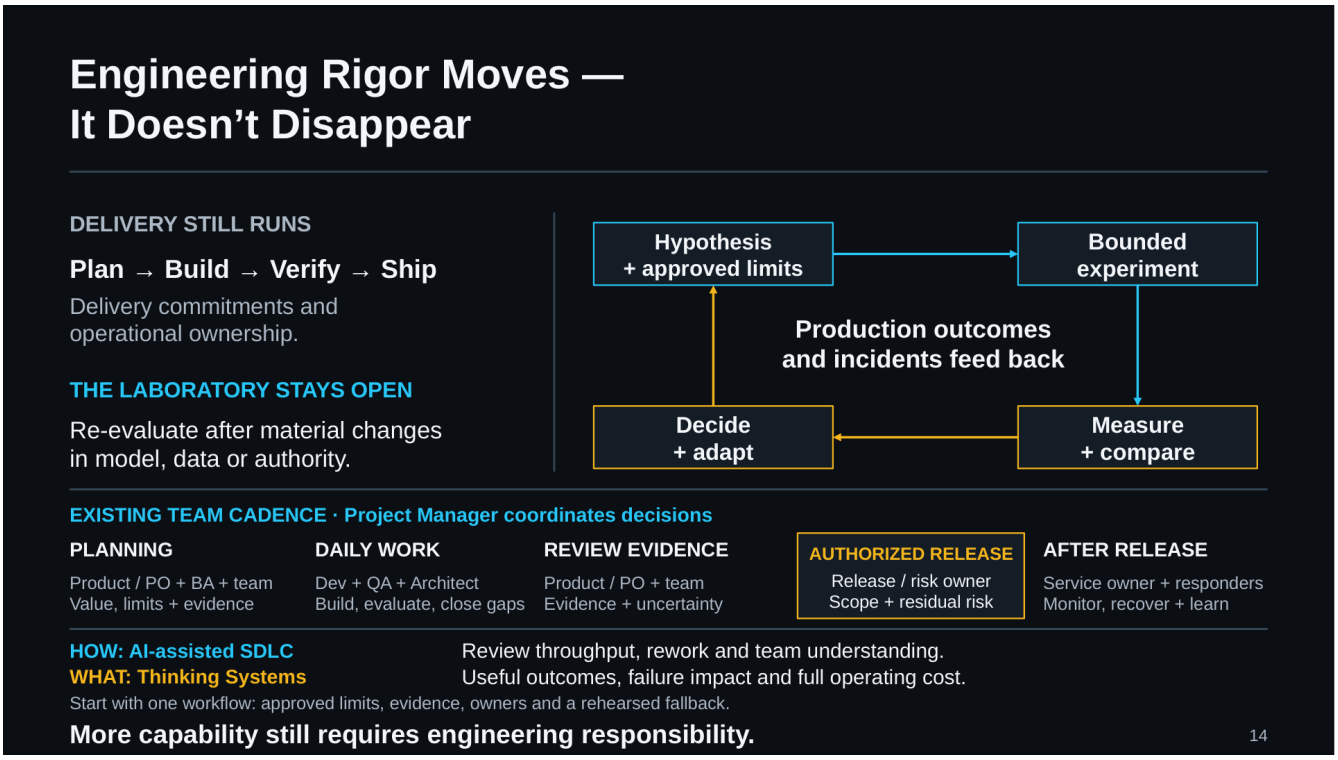
Establish	The decision it must support
Workflow and baseline	Which end-to-end outcome must improve?
Authority and prohibited results	What may the system do—and never do?
Verification, human capacity and full cost	Can we operate the necessary control?
Recovery and stop conditions	How will we contain a failed assumption?
Evidence for expansion or restriction	What result warrants the next step?

Rehearse both an agent unable to repair its error and an absent human controller. Demonstrate useful outcomes at acceptable full cost, with people able to investigate, decide and restore service. Where intervention would arrive too late, constrain the system before exposure. The observation period and adverse cases must fit the responsibility being delegated.

Evidence should change the adoption decision—including mine. A warning that refuses to change with evidence is not engineering either. Expand what works. Restrict or redesign what does not.

You do not owe a vendor your organisation as an experiment. You owe your customers a system you can operate.

“What if not?” becomes an acceptance criterion, a funded dependency and tested recovery—not an excuse for paralysis. Delivery obligations continue while we learn.



Production observations and incidents update assumptions and controls. Learning does not suspend delivery or service obligations. UA Online PMDay 2026 Autumn · slide 14 · Vitalii Oborskyi. [1]
Full-page slide 14 from Vitalii Oborskyi's v35 review deck. Source-deck and third-party credits preserved; publication rights pending.

Writing let ideas outlive their authors. AI expands the work we can delegate. The sum of our technology includes the systems, institutions and human capabilities needed to live with the result.

Build for the future you believe in. Keep the ability to act if that belief is wrong.

Further reading

The Subprime Code Crisis [9] and Uncertainty Architecture [20] develop the two research strands. They are my synthesis and developing architectural doctrine, not independent experimental confirmation of this essay.

Sources & references

Numbers beside named links connect claims to these sources. Source notes distinguish empirical findings, vendor-reported results, hypothetical examples and the author’s architectural argument.

1. Vitalii Oborskyi. AI Changes the Delivery System and the Product Itself. UA Online PMDay 2026 Autumn presentation. See the Slides (PDF) link at the beginning of this article.

Figures reproduce full slides 3, 5, 7, 9, 10, 12 and 14 from the v35 presentation copy. The event name follows the organiser's announcement. Original slide credits and notices remain applicable. Scenarios and proposed countermeasures are not empirical results.

2. Vitalii Oborskyi. UA Online PMDay 2026 Autumn talk recording (Ukrainian).
3. Jared Rubin. "Printing and Protestants: An Empirical Test of the Role of Printing in the Reformation." Review of Economics and Statistics 96(2), 270–286 (2014).
4. Stanisław Lem. Fantastyka i futurologia. Quotation selected in Wiktor Jaźniewicz's compilation, W strachu przed antymaterią. 1381 cytatów (Wydawnictwo Literackie, 2024).

   The Polish passage is reproduced under "TECHNOLOGIE JAKO TAKIE" in a publisher-supplied excerpt published by Instytut Spraw Obywatelskich on 22 July 2025, and attributed there to Fantastyka i futurologia—not Summa Technologiae. The English wording in the prologue is a translation prepared for this article from that Polish excerpt, not a quotation from a published English edition. No original-volume page number is supplied by the excerpt.
5. Cognition. "Introducing Devin, the first AI software engineer" (12 March 2024).

   The announcement also describes collaboration and human review; it is not a literal promise to replace all programmers immediately.
6. Council on Foreign Relations. CEO Speaker Series: Dario Amodei (10 March 2025).

   The 10 March 2025 forecast implied June–September 2025 for 90% and around March 2026 for essentially all code. These horizons have passed. This essay records the forecast, not a verified industry-wide share or a verdict on fulfilment. The same discussion retains human specification, design, integration and security responsibilities.
7. Mert Demirer, Leon Musolff and Liyuan Yang. "Writing Code vs. Shipping Code: Productivity Effects Across Generations of AI Coding Tools." NBER Working Paper 35275 (2026; September revision). · Author-hosted full text

   Table 6, printed p. 33: weeks 21–30, pre-period-normalised, winsorised event-study estimates. Cumulative multipliers: lines 1 + (234.3 + 957.5 + 1254.9)/100 = 25.467; commits 3.441; releases 1.288 (no separate async estimate). Outcome samples differ. Figure 1 rounds these to 25.5, 3.4 and 1.3. Marketplace figures: §7.2.3, January 2025–April 2026 cohorts. Observational working paper, not peer-reviewed.
8. Jessica Baolin and Nathen Harvey / DORA. "Balancing AI Tensions: Moving from AI Adoption to Effective SDLC Use" (10 March 2026; analyses the 2025 report).

   Survey associations, not randomised causal estimates.
9. Vitalii Oborskyi. The Subprime Code Crisis — independent research synthesis.

   Author-led synthesis, not independent experimental validation.
10. Anthropic. "Code Review" product and engineering announcement (2026).
11. Jay Parikh / Microsoft. "Introducing Command Line, and the new rules for builders" (2 June 2026).

    The opening supplies the old-playbook instruction; sections 8 and 10 supply the review and disposability headings, and section 3 the autopilot phrase. These passages are quoted separately, not combined into a statement that review itself is just an obsolete habit. First-pass review moves to agents, human architectural oversight remains, and the disposability argument is limited to well-bounded features. These are not a blanket claim that all code review is unnecessary.
12. Anthropic. "Solving Code Review at Anthropic" (15 October 2026).

    Internal June–September 2026 comparison, not an external causal or portability study. Reverted share and absolute reverted volume are different measures.
13. Shyam Agarwal, Hao He and Bogdan Vasilescu. "AI IDEs or Autonomous Agents? Measuring the Impact of Coding Agents on Software Development." arXiv:2601.13597v2 (27 January 2026).

    Table 2; §3.2 includes lines of code as a covariate. §4 notes limitations including isolated pre-treatment differences. Not a per-line or human-comprehension metric.
14. GitClear. The AI Code Quality & Maintainability Gap (2026).

    Block duplication versus the 2023 baseline; observational vendor report.
15. Alberto Bacchelli and Christian Bird. "Expectations, Outcomes, and Challenges of Modern Code Review." ICSE (2013), 712–721.
16. Lisanne Bainbridge. "Ironies of Automation." Automatica 19(6), 775–779 (1983). · Full text (IFAC)

    Foundational human-factors analysis, not an LLM deployment trial.
17. Ronny Kohavi, Thomas Crook, Roger Longbotham, Brian Frasca, Randy Henne, Juan Lavista Ferres and Tamir Melamed. "Online Experimentation at Microsoft" (2009). · Full text

    §5–5.1. The one-third result describes the authors' Microsoft experience, not a universal feature-success rate.
18. Joel Becker / METR. "Measuring the Self-Reported Impact of Early-2026 AI on Technical Worker Productivity" (11 May 2026).

    Convenience-sample self-reports. The 1.4–2× range reflects different question wording, not a confidence interval.
19. Debora Viana Thompson, Rebecca W. Hamilton and Roland T. Rust. "Feature Fatigue: When Product Capabilities Become Too Much of a Good Thing." Journal of Marketing Research 42(4), 431–442 (2005).

    Does not establish a human learning rate. The monthly-user/weekly-update example in this essay is hypothetical.
20. Vitalii Oborskyi. Uncertainty Architecture — open doctrine and pattern language.

    Author-led doctrine and pattern language; not independent empirical validation.
21. Uncertainty Architecture. "Uncertainty in the Controlled Object."

    Selected draft-normative definition. Consequential does not by itself mean high risk or production readiness.

22. Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz and Mario Fritz. “Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection.” arXiv:2302.12173v2 (5 May 2023).

    Demonstrates attacks through retrieved, attacker-influenced content in tested LLM-integrated applications. The account-recovery example in this essay is hypothetical, not a reported incident; the paper does not establish an attack success rate for current models.

23. OWASP Gen AI Security Project. “LLM01:2025 Prompt Injection.”

    Security guidance on direct/indirect injection, least privilege, trust boundaries and adversarial testing. Mitigation guidance is not a guarantee of immunity or evidence that this example's controls have been deployed.

24. NIST/SEMATECH. e-Handbook of Statistical Methods, §7.2.4.1, Confidence intervals for a proportion.

    Author calculations for the synthetic example: 1/200 gives a 95% Wilson interval of 0.0883%–2.7774%; 4/200 gives about 0.8%–5.0%. For zero failures, the exact one-sided 95% binomial upper bound is 1 − 0.05^(1/n); at n = 3,000 it is 0.0998%, close to the “rule of three”. Independence, representative sampling, reliable classification and stable conditions are assumptions—not release guarantees.

25. John C. Knight and Nancy G. Leveson. “An Experimental Evaluation of the Assumption of Independence in Multiversion Programming.” IEEE Transactions on Software Engineering SE-12(1), 96–109 (1986).

    Historical programming experiment, not a quantitative estimate of modern LLM error correlation.

26. Raja Parasuraman and Dietrich H. Manzey. “Complacency and Bias in Human Use of Automation: An Attentional Integration.” Human Factors 52(3), 381–410 (2010). · Abstract

    Research review of attention and automation-related decision errors in novice and expert users.

27. Uncertainty Architecture. “Control-Loop Capability Anatomy.”

    Selected draft-normative architectural framing, not field evidence that a particular human-control arrangement works.

28. NIST. Artificial Intelligence Risk Management Framework (AI RMF 1.0), Core (2023).

    Operational risk-management framework, not certification of the example system.

29. Alec Warner and Štěpán Davidovič, with Alex Hidalgo, Betsy Beyer, Kyle Smith and Matt Duftler. “Canarying Releases.” The Site Reliability Workbook, chapter 16 (2018).

    Defines canarying as partial, time-limited deployment plus evaluation to guide rollout. Its discussion of exposure, detection and isolation supports bounded learning, not a guarantee that all consequences can be reversed.

30. Manifesto for Agile Software Development. “Principles behind the Agile Manifesto” (2001).

    Connects agility with technical excellence, good design and sustainable development. Not a licence to omit consequences analysis or release without appropriate controls.

31. Stanisław Lem. Summa Technologiae (1964). · Quotation source: Bogna Konior, “Existential Technologies,” Antikythera.

    The responsibility quotation in part III is reproduced in Konior’s English translation. Her note 18 cites the Polish edition (Kraków: Wydawnictwo Literackie, 2020), p. 471; note 1 identifies the translations as her own. The quotation was checked in Konior’s essay, not independently against that book page. It must not be attributed to Joanna Zylinska’s published English translation. The sentence on delegated power is Oborskyi’s argument, not a quotation from Lem.

32. Marvin Muñoz Barón, Marvin Wyrich and Stefan Wagner. “An Empirical Validation of Cognitive Complexity as a Measure of Source Code Understandability.” ESEM (2020). · DOI

    Meta-analysis of ten studies, 427 code snippets and approximately 24,000 evaluations. Finds associations with comprehension time and subjective ratings; correctness and physiological results are mixed. The metric weights nested control flow. It does not establish a universal nonlinear relationship between a percentage change in the score and human cognitive load.

33. Luigi Lavazza, Sandro Morasca and Marco Gatto. “An empirical study on software understandability and its dependence on code characteristics.” Empirical Software Engineering 28, 155 (2023).

    Study of three master's students performing maintenance tasks on 32 methods from two applications. Structural metrics correlate with task time, but prediction accuracy is limited; Cognitive Complexity does not improve it in this setting. Small sample. The discussion of disproportionate effort from interacting conditions and dependencies is an engineering explanation of a risk, not a measured scaling law from this study.

How to cite this draft

Oborskyi, V. (2026). The Sum of Our Technology: AI Changes Both Sides of Software Engineering. Editorial draft R18, 20 October 2026.

Unpublished author-review edition. Cite the revision; no DOI or final publication URL has been assigned.

```
@misc{oborskyi2026sumR18,
  author = {Oborskyi, Vitalii},
  year = {2026},
  title = {The Sum of Our Technology:
           AI Changes Both Sides of Software Engineering},
  note = {Editorial draft R18, 20 October 2026; unpublished}
}
```

Text © 2026 Vitalii Oborskyi. Presentation and third-party credits remain applicable. Cover 02: AI-generated editorial illustration selected by the author.